

Database Systems

User Defined Functions and Triggers

Nicola Saccomanno

`nicola.sacomanno@uniud.it`

Bachelor degrees in Internet of things, Big Data, Machine Learning, & Computer Science
DMIF, University of Udine

A.Y. 2025/2026

User-Defined Functions

- ▶ The SQL standard defines a set of functions for data manipulation
 - ▶ `CURRENT_TIMESTAMP` returns the current moment
 - ▶ `||` concatenates two strings
 - ▶ Aggregate Functions
 - ▶ Etc...
- ▶ Users can also define their own functions (user-defined function, UDF)
- ▶ UDFs are stored in the DBMS server, like other database objects
- ▶ Functions can be written in SQL or another language
- ▶ The syntax used (and possible allowed usages) by various DBMSs is not uniform
- ▶ Note: the following examples use the syntax of PostgreSQL

Functions in PL/pgSQL Language

```
CREATE or REPLACE FUNCTION <name(parameters)>
RETURNS <type>
LANGUAGE plpgsql AS $$
    DECLARE
        <variable declarations>
    BEGIN
        <instructions>
    END;
$$;
```

- ▶ The body of the function (between \$\$) is technically a string
- ▶ The function returns a value using a RETURN statement
- ▶ Functions with the same name and return value can be defined, provided they have different parameters (overloading)
- ▶ The result of a query is assigned to a variable with SELECT INTO

UDF a possible general structure

```
CREATE FUNCTION example_function(  
    <param1 type1>, <param2 type2>  
) RETURNS <type> LANGUAGE plpgsql AS $$  
    DECLARE  
        <variable declaration>  
    BEGIN  
        SELECT INTO <variable>  
            <query>  
        IF <condition> THEN  
            <action>  
        ELSE  
            <alternative action>  
        END IF;  
        RETURN <variable>;  
    END;  
$$;
```

- ▶ Note: The SELECT INTO statement is used to assign the result of a query to a variable
- ▶ The RETURN statement is used to return the result

UDF example

```
CREATE or REPLACE FUNCTION dep_count(  
    in_depname varchar(20)  
) RETURNS integer LANGUAGE plpgsql AS $$  
    DECLARE  
        d_count integer;  
    BEGIN  
        SELECT count(*) into d_count  
        FROM department, employee  
        WHERE dno = dnumber  
              and dname = in_depname;  
        RETURN d_count;  
    END;  
    $$;
```

- ▶ This UDF calculates the number of employees in a department, given the department name as input
- ▶ The result of the query (SELECT statement), which - in this case - is a single number from the count operation, is stored in the d_count variable, that is the output of the UDF

-- Usage example

```
select dname, dep_count(dname) from department;
```

Input and output parameters

```
CREATE or REPLACE FUNCTION dep_count(  
    in in_depname varchar(20),  
    out d_count integer  
) LANGUAGE plpgsql AS $$  
    BEGIN  
        SELECT count(*) into d_count  
        FROM department, employee  
        WHERE dno = dnumber  
              and dname = in_depname;  
    END;  
$$;
```

- ▶ In the presence of OUT parameters, the RETURN clause is redundant
- ▶ A function can have multiple OUT parameters (in such cases, the returned object is of type RECORD)
- ▶ LANGUAGE plpgsql is optional (it is the default)

Multiple output parameters

```
CREATE FUNCTION sum_product(x int, y int, out sum int, out prod int)
LANGUAGE plpgsql AS $$
BEGIN
    sum := x + y;
    prod := x * y;
END;
$$;
```

```
select sum_product(3,4);
```

```
sum_product
```

```
-----
```

```
(7, 12)
```

```
select * from sum_product(3,4);
```

```
sum | prod
```

```
-----+-----
```

```
7 | 12
```

- ▶ Example using multiple parameters as output
- ▶ According to where we use the UDF in the external query, the behaviour changes (record/tuple of values vs relation)

Alternative solution

```
CREATE TYPE sum_prod as (sum int, prod int);
```

```
CREATE or REPLACE FUNCTION sum_product(  
    x int, y int  
)
```

```
RETURNS sum_prod
```

```
LANGUAGE plpgsql AS $$
```

```
    DECLARE
```

```
        result record;
```

```
    BEGIN
```

```
        result := (x + y, x * y);
```

```
        return result;
```

```
    END;
```

```
$$;
```

- ▶ An alternative solution is to define a new TYPE whose structure reflects the desired output of the UDF
- ▶ Then, we can use such a type as a return type of the UDF
- ▶ In the UDF body, we can make use of a general record type, to which we can assign, following valid structure and domains with respect to our target type, the desired result

Functions are parameterized views: definition

```
CREATE or REPLACE FUNCTION female_dep(dep_name varchar(20))
RETURNS TABLE (
    Lname varchar(50),
    Fname varchar(50),
    SSN char(10)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
    SELECT Lname, Fname, SSN
    FROM employee, department
    WHERE dno = dnumber and
           dname = dep_name and
           sex = 'F';
END;
$$;
```

- ▶ The return value can also be a table
- ▶ The UDF, given a department name, return a table of all the female employees working in such a department
- ▶ By using RETURN QUERY we specify that the output of the UDF is the result of the evaluation of the SQL query specified next
- ▶ This is a generalization of the view mechanism (i.e., here we have parameters that define our “view”)

Functions are parameterized views: usage

The previously defined function can be used as follows:

```
select * from female_dep('Department 1');
```

Lname	Fname	SSN
White	Mary	WHTMRY85E57Z114V

```
select Lname from female_dep('Department 3') where Fname = 'Anne';
```

Lname
Mountbatten
Coon

Functions for constraint verification

- ▶ A function can be invoked inside a CHECK clause (constraint... check...)
- ▶ Such a function can be used to implement complex constraints that can be violated only during insertion or update (but not by deletions)
- ▶ Each CHECK clause is verified immediately after the insertion or update of each record and is satisfied when the evaluated expression is TRUE or UNKNOWN (be careful about NULL values!!!)
- ▶ Important limitations in PostgreSQL:
 - ▶ a CHECK constraint must depend only on the row being inserted or updated; it must not enforce cross-row or cross-table conditions. A function used in a CHECK should behave as immutable with respect to that row
 - ▶ CHECK constraints are not deferrable
- ▶ Triggers (next topic) provide an alternative, more flexible, and in some cases more efficient mechanism for verifying integrity constraints

Functions for constraint verification: (unsafe) example

```
CREATE or REPLACE FUNCTION check_sup(emp char(10))
RETURNS boolean LANGUAGE plpgsql AS $$
    DECLARE
        ok boolean;
    BEGIN
        raise notice
            'Check supervisor of %', emp;
        SELECT E.Dno = S.Dno into ok
        FROM employee E join employee S
            on E.Super_SSN = S.SSN
        WHERE E.SSN = emp;
        RETURN ok;
    END;
$$;
```

▶ Constraint: an employee and its supervisor must belong to the same department

▶ The function can be called inside a CHECK clause, but this is not a sound way to enforce a cross-row constraint in PostgreSQL

▶ The function reads other rows of employee, while CHECK constraints should depend only on the row being inserted or updated

▶ Later updates or deletions of the supervisor info do not automatically revalidate the constraint

▶ Therefore this solution may appear to work, but it is unsafe; a trigger is the proper mechanism here

```
alter table employee add constraint supervision check(check_sup(SSN));
```

UDF: perform and found operators

```
CREATE FUNCTION example_function(  
    <param1 type1>, <param2 type2>  
) RETURNS <type> LANGUAGE plpgsql AS $$  
    DECLARE  
        <variable declarations>  
    BEGIN  
        PERFORM *  
            <query without SELECT>;  
        IF FOUND THEN  
            <action>  
        ELSE  
            <alternative action>  
        END IF;  
        RETURN <variable>;  
    END;  
$$;
```

- ▶ Standard SELECT operator can not be used within a UDF.
- ▶ PERFORM can be used instead, which simply execute the query. SELECT... INTO... can be used to store the outcome into a variable
- ▶ FOUND is a boolean variable that is automatically set depending on the outcome of the last executed query within the UDF (True iff query returned/modified/deleted something, False iff query results was empty)

Procedural Extensions of SQL

- ▶ Variable declarations (DECLARE)
- ▶ Assignments (e.g., `variable := value`; `n := n + 1`)
- ▶ Conditional constructs (IF...THEN...ELSE, CASE)
- ▶ Iterative constructs (LOOP, FOR, WHILE)
- ▶ Exception handling (RAISE EXCEPTION)
- ▶ Refer to §40.1–40.6 of the PostgreSQL manual
- ▶ It is possible to write functions and procedures in an external language (C, Java, PHP, Python, Ruby, R, Scheme, ...)
- ▶ For some languages (e.g., Java), functions can be executed in a sandboxed environment with restricted memory access

UDF: Advantages

- ▶ Simplification of applications through the sharing of general-purpose code among different applications
- ▶ Uniform semantics for certain operations on the database
- ▶ Centralized control of integrity constraints not expressible in SQL (with caveats!)
- ▶ Reduction of network traffic: the client only needs to send a remote procedure call instead of a sequence of SQL instructions
- ▶ Data security: access to data can be allowed only through defined procedures and functions, without permitting users direct access to the tables (similar to the mechanism of views)

UDF: Disadvantages

- ▶ Limit in portability (each DBMS has its own syntax)
- ▶ Extensive use of UDFs is not considered good practice by some in the software engineering community (debatable)
- ▶ Some software design methodologies do not support the development of such procedures
 - ▶ Especially in the context of “agile” methodologies, which envisage rapid software evolution
- ▶ Greater expertise required: DBAs (database administrators) and programmers are professional figures with different skills
- ▶ Often DBAs and software developers are in separate teams, leading to management and communication issues between workgroups

Trigger

- ▶ An action automatically executed by the system as a result of a database modification operation (insert, update, or delete) on a specific table
- ▶ A trigger is activated independently of the user or application performing the operation
- ▶ The activation of a trigger causes the execution of a UDF
- ▶ Simplified **syntax**:

```
CREATE TRIGGER <name> { BEFORE | AFTER } <event>
ON <table>
FOR EACH { ROW | STATEMENT }
[ WHEN (<condition>) ]
EXECUTE FUNCTION <function()>;
```

- ▶ Each DBMS has its own syntactic variations

Characteristics of Triggers

- ▶ The event can be INSERT, UPDATE, or DELETE on a specific table
- ▶ A trigger can be activated before (BEFORE) or after (AFTER) the event
- ▶ A condition for activation (WHEN) can be specified
- ▶ A trigger can be activated once per event (FOR EACH STATEMENT) or once for each record modified by the event (FOR EACH ROW)
- ▶ It can be executed immediately after the activating event, or be postponed until the end of the current transaction (deferred execution)
 - ▶ In PostgreSQL, only AFTER triggers are deferrable
- ▶ If multiple triggers are activated by the same event, the order of execution depends on the implementation
 - ▶ It may coincide with the order of definition
 - ▶ It may be automatically determined by the system
 - ▶ It may be defined by the user
 - ▶ In PostgreSQL, triggers are executed in alphabetical order

Trigger: Observations

- ▶ A trigger can perform data checks and modify the state of the database
 - ▶ Event logging (auditing, replication, synchronization of duplicated data, management of materialized views, etc.)
- ▶ It is preferable to use other mechanisms, if possible
 - ▶ For example, the `ON DELETE CASCADE` clause for cascading deletions
 - ▶ Built-in functionality for replication, auditing, and materialized views
- ▶ A trigger can activate others in a cascade, raising issues of consistency and termination
 - ▶ For instance, trigger *A* activates trigger *B*, which reactivates trigger *A*
 - ▶ Some systems (e.g., Oracle) prohibit recursive activations
- ▶ Unintended activations should be avoided
 - ▶ For example, in backup recovery procedures or replication through SQL command replay (statement-based replication)

Functions for triggers in PL/pgSQL

```
CREATE or REPLACE FUNCTION <function_name()>
RETURNS trigger
LANGUAGE plpgsql AS $$
    DECLARE
        <variable declaration>
    BEGIN
        <instructions>
        RETURN <NULL or record>
    END;
$$;
```

- ▶ The function does not have any parameter
- ▶ A special return type, returns trigger, is used
- ▶ A return statement must always be present

PL/pgSQL Triggers: NEW and OLD variables

- ▶ Two special variables are automatically available when using triggers:
 - ▶ NEW - usable in triggers activated by INSERT or UPDATE operations, representing the record that is being inserted or updated
 - ▶ OLD - usable in triggers activated by DELETE or UPDATE operations, representing the record that is being deleted or modified
- ▶ NEW and OLD are of type RECORD
- ▶ A field/attribute X of a record is denoted, for instance, by NEW.X or OLD.X
- ▶ The use of these variables only makes sense in functions invoked by triggers defined FOR EACH ROW
- ▶ Note that NEW and OLD can be used also in the WHEN clause of the trigger

PL/pgSQL Triggers: Return Value

- ▶ A function fired by a trigger must return a value via the RETURN statement
 - ▶ Either NULL
 - ▶ Or a record/row having exactly the structure of the table the trigger was fired for
- ▶ BEFORE - FOR EACH ROW triggers can return NULL to signal the trigger manager to skip the rest of the operation for this row
 - ▶ Subsequent triggers are not fired
 - ▶ INSERT/UPDATE/DELETE has no result for the current row
- ▶ If a non-null value is returned then the operation proceeds. Such a value can be:
 - ▶ A row value different from the original value of NEW, effectively altering the row that will be inserted or updated (e.g., replace single values directly in NEW `new.age := new.age + 1`; or build a complete new record/row to return)
 - ▶ Directly NEW, if the trigger function wants the triggering action to succeed normally without altering the values of the affected row (or a value equal thereto)
 - ▶ On DELETE, the returned value has no direct effect, but it has to be non-null to allow the trigger action to proceed (conventionally, return OLD)
- ▶ If the trigger is defined as AFTER or FOR EACH STATEMENT, the return value is ignored, so RETURN NULL can be used
- ▶ For further details, refer to Chapter 36 of the PostgreSQL manual

Trigger: example

```
CREATE or REPLACE FUNCTION check_num_dep()  
RETURNS trigger LANGUAGE plpgsql AS $$  
    DECLARE  
        n integer;  
    BEGIN  
        SELECT count(*) INTO n  
        FROM Employee E  
        WHERE NEW.dno = E.dno;  
  
        IF n >= 6 THEN  
            raise notice 'Max amount reached';  
            RETURN null; -- Skip the op.  
        END IF;  
  
        RETURN NEW;  
    END;  
$$;
```

- ▶ Let's build a function to supports a trigger that checks, at each insert and update operation, the following constraint:
"No department can have more than six employees"
- ▶ Intuitively, each time we insert a new employee, or we change the dno attribute for an employee's record, we need to check if such operation leads to a violation of the constraint

Trigger: Example (continued)

The previous function does not constitute the trigger in its entirety. It should be invoked:

- ▶ When a new employee is inserted
- ▶ When changing an employee's department (because an employee cannot be moved to a department that already has the maximum number of employees)

```
CREATE TRIGGER check_dept_capacity  
BEFORE INSERT OR UPDATE ON Employee  
FOR EACH ROW  
EXECUTE FUNCTION check_num_dep();
```


How can we model A (X,Y) R (1,N) B cases using triggers?

- ▶ In the translation from the ER to the logical schema, we said that the mandatory participation of entities on the “many” side of one-to-many and many-to-many relationships cannot be enforced using core logical-model constructs
- ▶ To model a constraint correctly using triggers, we first need to ask ourselves:
What insert, update, or delete operations and on which tables can lead to a violation of my constraint?
- ▶ Example: A professor must teach at least one course, but they can teach more than one; each course is taught by exactly one professor.
 - ▶ Professor(SSN, ...)
 - ▶ Course(ID, professor, ...), VNN: {professor}, FK: {Course.professor→Professor.SSN}
- ▶ Which operations on which tables can cause violations?
 - ▶ Insertion on Professor: when I insert a new professor, I must also associate at least one course with him/her
 - ▶ Update and delete on Course: if I delete a course or change its associated professor, I must ensure that the professor originally associated with that course does not end up with no assigned courses

How can we model A (X,Y) R (1,N) B cases using triggers? I

Example: Insert on professor - a possible solution

- ▶ First, we define a trigger on Professor that checks, for every inserted professor, whether they have an associated course
- ▶ Since Course.professor must reference an existing professor, one possible solution is to defer the foreign-key check when inserting a new course-professor pair
 - ▶ Insert a new course that references professor X, with the foreign-key check deferred
 - ▶ Insert professor X; the trigger will then check whether Course contains a tuple referencing X
 - ▶ The transaction ends, and the deferred constraints are checked
- ▶ In PostgreSQL it could be useful to use

`RAISE EXCEPTION '...';`

rather than return `RETURN null`; in this way, instead of (silently) skipping the row operation, we raise an actual integrity error for the transaction

How can we model A (X,Y) R (1,N) B cases using triggers? II

Example: Insert on professor - a possible solution

```
CREATE or REPLACE FUNCTION
```

```
check_prof_has_course()
```

```
RETURNS trigger LANGUAGE plpgsql AS $$
```

```
BEGIN
```

```
    PERFORM *
```

```
    FROM Course
```

```
    WHERE Course.professor = NEW.SSN;
```

```
    IF FOUND THEN
```

```
        RETURN NEW;
```

```
    ELSE
```

```
        RAISE EXCEPTION 'No course associated  
        to prof' -- Roll-back/abort;
```

```
    END IF;
```

```
END;
```

```
$$;
```

```
CREATE TRIGGER
```

```
check_prof_integrity_1
```

```
BEFORE INSERT ON PROFESSOR
```

```
FOR EACH ROW
```

```
EXECUTE FUNCTION
```

```
check_prof_has_course();
```

How can we model A (X,Y) R (1,N) B cases using triggers? I

Example: Delete and update on course - a possible solution

- ▶ We need a trigger that checks whether we are deleting the last course assigned to a professor
- ▶ We also need a similar trigger that checks whether changing the professor assigned to a course would leave the previous professor with no courses

How can we model A (X,Y) R (1,N) B cases using triggers? II

Example: Delete and update on course - a possible solution

```
CREATE or REPLACE FUNCTION
```

```
check_last_course_del()
```

```
RETURNS trigger LANGUAGE plpgsql AS $$
```

```
DECLARE
```

```
    n integer;
```

```
BEGIN
```

```
    SELECT count(*) INTO n
```

```
    FROM Course
```

```
    WHERE Course.professor = OLD.professor;
```

```
    IF n = 1 THEN
```

```
        RAISE EXCEPTION 'Removing last  
        course for a prof';
```

```
    END IF;
```

```
    RETURN OLD;
```

```
END;
```

```
$$;
```

```
CREATE TRIGGER
```

```
check_prof_integrity_2
```

```
BEFORE DELETE ON Course
```

```
FOR EACH ROW
```

```
EXECUTE FUNCTION
```

```
check_last_course_del();
```

How can we model A (X,Y) R (1,N) B cases using triggers? III

Example: Delete and update on course - a possible solution

```
CREATE or REPLACE FUNCTION
```

```
check_last_course_upd()
```

```
RETURNS trigger LANGUAGE plpgsql AS $$
```

```
DECLARE
```

```
    n integer;
```

```
BEGIN
```

```
    SELECT count(*) INTO n
```

```
    FROM Course
```

```
    WHERE Course.professor = OLD.professor;
```

```
    IF n = 1 THEN
```

```
        RAISE EXCEPTION 'Removing last  
        course for a prof';
```

```
    END IF;
```

```
    RETURN NEW;
```

```
END;
```

```
$$;
```

```
CREATE TRIGGER
```

```
check_prof_integrity_3
```

```
BEFORE UPDATE ON Course
```

```
FOR EACH ROW
```

```
WHEN
```

```
(NEW.professor <> OLD.professor)
```

```
EXECUTE FUNCTION
```

```
check_last_course_upd();
```